Lab 3: Recursion & Sorting

Total Points: 50

Objectives

- Practicing in **recursive thinking**.
- Writing **recursive function**.
- Practicing on Quicksort and Merge Sort

Part 1 - Recursion

Question 1 - [8 points] The following codes are using loops to solve the problem, rewrite them using the condition of loop as base case and implement the loop with recursion.

```
int sumDigits(int num){
     int sum = 0;
     while (num > 0) {
          int digit = num % 10;
          sum += digit;
          num = num / 10;
     }
     return sum;
}
```

Question 2 - [10 points] Write a **recursive function** that displays all the single-digit values stored in an array of integers on the screen. For example, if the input array includes `{12, -234, -5, 1, 345, 4, 8976, 0}`, the function displays `0, 4, 1, -5,` on the screen. A recursive algorithm for this function is given as below:

- If an array is empty, there is nothing to display on the screen.
- If the array has one or more elements, the function first checks the last value in the array to see if it should be displayed on the screen or not.
- It then **recursively displays** (recursive call) all the single-digit values of the array **excluding the last element** in the array.

You **must** use the following main function to test your function. Make sure your function header matches with the following function call.

```
int main(){
  int someData [] = {12, -234, -5, 1, 345, 4, 8976, 0};
  displaySingleDigits(someData,8); // prints 0, 4, 1, -5,
  return 0;
}
```

Part 2 – Merge Sort

Question 3 - [10 points] Change the **merge()** function (in slide7 of the lecture notes) to work with **mergeSort()** function.

```
void merge(const int data1[],int n,const int data2[],int m,int data3[]){
    int i, j, k;
    for (i = j = k = 0; i < n && i < m;    ){
        if (data1[i] < data2[j])
            data3[k++] = data1[i++];
        else
            data3[k++] = data2[j++];
    }
    while (i < n) {
      data3[k++] = data1[i++];
    }
    while (j < m) {
      data3[k++] = data2[j++];
    }
}
// the function that sorts arr[strat...end] using merge()
void mergeSort(int arr[], int start, int end) {
   if (start >= end)   // Base case: 1 or 0 elements
            return;
   int mid = start + (end - start) / 2; // step 1
   mergeSort(arr, start, mid);      //step 2
   mergeSort(arr, mid + 1, end);    //step 2
   merge(arr, start, mid, end);     //step 3
}
```

Part 3 – Quick Sort

Question 4 - [10 points] Adjust the following **`partition()`** function (explained in lecture notes) to work with **`quicksort()`** function listed below.

```
int partition(int arr[], int size) {
    int pivot = arr[0];
    int low = 1;
    int high = size - 1;
    while (low < high) {
        // Move low to the right
        while (low < size && arr[low] <= pivot)
                low++;
        // Move high to the left
        while (arr[high] > pivot)
                high--;
        // If low < high, swap elements
        if (low < high)
                swap(arr[low], arr[high]);
    }
    // Step 4: Swap pivot with element before low
    swap(arr[0], arr[low-1]);
    // Return final position of pivot
    return low-1;
}
```

```
// Quick Sort function
void quickSort(int arr[], int size) {
    quickSort(arr, 0, size - 1);
}
// Quick Sort Helper function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // Partition the array
        int pivotIndex = partition(arr, low, high);
        // Recursively sort left and right parts
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}
```

Question 5 - [4 points] Write a function named **partitionV2** like the partition function in the above code (or described in the lecture notes), that takes **the last element as its pivot**.

Part 3 – Comprehension

Question 6 – [8 points] Choose the best sorting algorithm among selection, insertion, bubble, merge, and quick sort, along with a clear justification, for the following scenarios:

a) When the data are sorted in the right order -
b) When the data are almost sorted in the opposite order -
c) When the data are sorted in the in the opposite order -
d) When the data are randomly arranged -

Answer this question as a comment in the .cpp file.

Submission

Submit a file named **programming.cpp** in the Brightspace before the due date.